TECHNICAL REPORT

Aluno: Antônio Kauê e José Basílio

1. Introdução

Neste relatório, o dataset usado diz respeito à previsão de Acidente Vascular Cerebral (AVC) em indivíduos com base em características demográficas e de saúde, sendo uma das principais causas de mortalidade no mundo. Este conjunto de dados, composto por informações de pacientes, oferece a oportunidade de investigar fatores como idade, gênero e condições clínicas que influenciam o risco de AVC. A análise busca não apenas compreender esses fatores, mas também estabelecer bases para modelagens preditivas que possam contribuir para intervenções precoces.

2. Observações

Durante a execução da AV2, algumas dificuldades dificultaram bastante, uma delas foi a de analisar o Dataset, onde a gente teve dificuldade de interpretar os dados que o Dataset fornecia. Juntamente com esse problema, na criação e entendimento dos códigos tivemos dificuldade também.

3. Resultados e discussão

Questão 1

Na AV1 você realizou uma análise do KNN para descobrir o melhor valor de k e também a forma mais adequada de determinar a métrica de distância. Utilizando o GridSearchCV que está disponível na scikit-learn, realize o mesmo processo para descobrir qual a melhor parametrização no processo de classificação do seu dataset. Gere os dados de acertos completos e faça uma análise do resultado. Verifique se o resultado obtido na AV1 é o mesmo que você obteve nesta AV2.

Esse código usa o **Random Forest** e o **GridSearchCV** para encontrar a melhor configuração do modelo de classificação de AVCs no dataset.

Random Forest é um algoritmo de aprendizado de máquina supervisionado, usado para classificação e regressão. Ele funciona criando várias árvores de decisão e combinando seus resultados para obter uma previsão mais precisa e robusta.

GridSearchCV é uma ferramenta do Scikit-Learn usada para encontrar os melhores hiperparâmetros de um modelo, testando várias combinações de forma automática.

Importação das bibliotecas necessárias.

```
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.ensemble import RandomForestClassifier
```

pandas: Para manipular os dados, carregar o dataset e realizar operações como remoção de colunas e tratamento de valores ausentes.
sklearn.model_selection.train_test_split: Para dividir os dados em treino e teste.
sklearn.model_selection.GridSearchCV: Para encontrar os melhores hiperparâmetros do modelo através de validação cruzada.
sklearn.preprocessing.LabelEncoder: Para converter variáveis categóricas em valores numéricos.
sklearn.preprocessing.StandardScaler: Para normalizar os dados e garantir que todas as variáveis tenham a mesma escala.
sklearn.ensemble.RandomForestClassifier: Para aplicar o algoritmo de classificação baseado em múltiplas árvores de decisão.

Removendo colunas e tratando valores ausentes no bmi

```
df.drop(columns=['id'], inplace=True)

df['bmi'] = df['bmi'].fillna(df['bmi'].mean())
```

Removemos a coluna “id”, pois ela não é útil para o modelo, além disso substituímos os valores ausentes do IMC (bmi) pela média da coluna.

Convertendo variáveis categóricas em número e separando os dados em entrada (X) e saída (y)

```
categorical_cols = ['gender', 'ever_married', 'work_type', 'Residence_type', 'smoking_status']
label_encoders = {}

for col in categorical_cols:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
    label_encoders[col] = le

X = df.drop(columns=['stroke'])
y = df['stroke']
```

Utilizamos o LabelEnconder para transformar os valores de texto em número pois o modelo só trabalha com números.

Logo após, separamos os dados para as variáveis X, que contém todas as colunas menos a de “stroke”, e na variável y, contém a coluna stroke, que é o alvo a ser previsto.

Dividindo os dados em teste e treino e normalizando-os

```
X_train, X_test, y_train, y_test = train_test_split( *arrays: X, y, test_size=0.2, random_state=42, stratify=y)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Os dados foram divididos da seguinte forma: 80% usado para treino e 20% destinado para teste.

Parâmetros	O que faz?	Função no código
test_size=0.2	Usa 20% dos dados para teste e 80% para treino.	Mantém a proporção das classes no treino e teste.
random_state=42	Mantém a mesma divisão de dados em cada execução.	Permite repetir os experimentos com os mesmos resultados.
stratify=y	Mantém a proporção das classes no treino e teste.	Evita que o modelo aprenda de forma desbalanceada.

A normalização foi responsável por deixar todas as variáveis na mesma escala para evitar que a uma inflencie mais que outra.

Definindo o modelo Random Forest

```
rf = RandomForestClassifier(random_state=42)
```

Define o Random Forest, um conjunto de várias árvores de decisão.

Definindo os hiperparâmetros para otimização

```
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}
```

Responsável por definir diferentes combinações de configurações para testar no modelo.

Configurando e rodando o GridSearchCV e Exibir os melhores parâmetros

```
grid_search = GridSearchCV(rf, param_grid, cv=5, scoring='accuracy', n_jobs=-1, verbose=1)

grid_search.fit(X_train, y_train)

print("Melhores hiperparâmetros:", grid_search.best_params_)
```

O GridSearchCV testa todas as combinações de hiperparâmetros e escolhe a melhor.

Parâmetros	O que faz?	No código
rf	Modelo a ser otimizado	RandomForestClassifier
param_grid	Lista de hiperparâmetros a testar	{'n_estimators': [50, 100, 200], 'max_depth': [None, 10, 20], ...}
cv=5	Validação cruzada com 5 divisões	Garante avaliação mais confiável

scoring='accuracy'	Usa acurácia como métrica de desempenho	Escolhe o modelo com mais acertos
n_jobs=-1	Usa todos os processadores disponíveis	Acelera os testes
verbose=1	Mostra mensagens de progresso	Ajuda a acompanhar a execução

Logo após é mostrada a melhor configuração encontrada.

Avaliando no conjunto de teste

```
y_pred = grid_search.best_estimator_.predict(X_test)
accuracy = (y_pred == y_test).mean()
print("Acurácia no conjunto de teste:", accuracy)
```

Responsável por fazer previsões no conjunto de teste, além de calcular a acurácia, ou seja, quantas previsões estão corretas. E após isso, exibir o valor da acurácia.

Resultado da Questão

```
Fitting 5 folds for each of 81 candidates, totalling 405 fits
Melhores hiperparâmetros: {'max_depth': None, 'min_samples_leaf': 4, 'min_samples_split': 2, 'n_estimators': 50}
Acurácia no conjunto de teste: 0.9510763209393346
```

O GridSearchCV testou 81 combinações de hiperparâmetros (cada uma testada 5 vezes, totalizando 405 execuções)

A melhor configuração encontrada foi:

Árvores sem limite de profundidade.

Pelo menos 4 amostras por folha.

Uma divisão só ocorre se houver pelo menos 2 amostras.

Usa 50 árvores no Random Forest.

O modelo foi avaliado no conjunto de teste e teve uma acurácia de 95,1%, ou seja, previu corretamente 95,1% dos casos.

QUESTÃO 2

Sem considerar a coluna alvo do seu dataset, faça uma análise para descobrir a quantidade ideal de clusters que deve ser feita. Considerem usar o método do cotovelo e da silhueta.

Importação das bibliotecas necessárias

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
```

pandas: Para manipular os dados.

numpy: Para operações matemáticas.

matplotlib.pyplot: Para criar gráficos.

sklearn.cluster.KMeans: Para aplicar o algoritmo de clustering (agrupamento).

sklearn.preprocessing.StandardScaler: Para normalizar os dados.

sklearn.metrics.silhouette_score: Para medir a qualidade dos agrupamentos.

Carregamento e limpeza dos dados

```
# Carregar o dataset
df = pd.read_csv(r"C:\Users\Kauem\Downloads\healthcare-dataset-stroke-data.csv")

df.drop(columns=['id', 'stroke'], inplace=True)

df['bmi'] = df['bmi'].fillna(df['bmi'].mean())
```

É Carregado o dataset no pandas em forma CSV

Removemos as colunas “id” pois não influencia na análise dos clusters(apenas um identificador. Removemos também a coluna alvo **“stroke”.**

É feita uma média para preencher os valores ausentes na coluna “bmi”

Transformação das variáveis categóricas e numéricas

```
categorical_cols = ['gender', 'ever_married', 'work_type', 'Residence_type', 'smoking_status']
df = pd.get_dummies(df, columns=categorical_cols)
```

Usamos a função **pd.get_dummies** para transformar variáveis categóricas em colunas numéricas

Se tivermos a coluna gender com valores ["Male", "Female", "Other"], ela será transformada em três colunas: gender_male, gender_female e gender_other cada uma com 0 ou 1.

Algumas colunas têm valores textuais, como “Male” e “Female”. Dessa forma a função converte essas colunas em números, como por exemplo, “Male” vira 1 e “Female” vira 0.

Normalização dos dados

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df)
```

Essa parte é usada para normalizar os dados para que todas as colunas tenham valores comparáveis, pois os valores do dataset variam bastante.

Método cotovelo e silhueta

```
inertia = []
silhouette_scores = []
K_range = range(2, 11)

for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    inertia.append(kmeans.inertia_)

    silhouette_avg = silhouette_score(X_scaled, kmeans.labels_)
    silhouette_scores.append(silhouette_avg)
```

É criada uma lista para armazenar os resultados de cada tentativa de agrupamento, e logo após testamos valores de k, de 2 até 10 grupos.

Após isso, criamos um loop para testar diferentes valores de k, dessa forma:

1- Para cada k, aplicamos um K-Means.

2- Salvamos a inércia que mede a dispersão dos pontos no cluster.

3- E por fim calculamos o coeficiente de silhueta que mede a quantidade de clusters.

Plotagem dos gráficos

```
plt.figure(figsize=(12, 5))
plt.subplot( *args: 1, 2, 1)
plt.plot( *args: K_range, inertia, marker='o', linestyle='--')
plt.xlabel('Número de Clusters (k)')
plt.ylabel('Inércia')
plt.title('Método do Cotovelo')

plt.subplot( *args: 1, 2, 2)
plt.plot( *args: K_range, silhouette_scores, marker='o', linestyle='--', color='red')
plt.xlabel('Número de Clusters (k)')
plt.ylabel('Coeficiente de Silhueta')
plt.title('Análise de Silhueta')

plt.show()
```

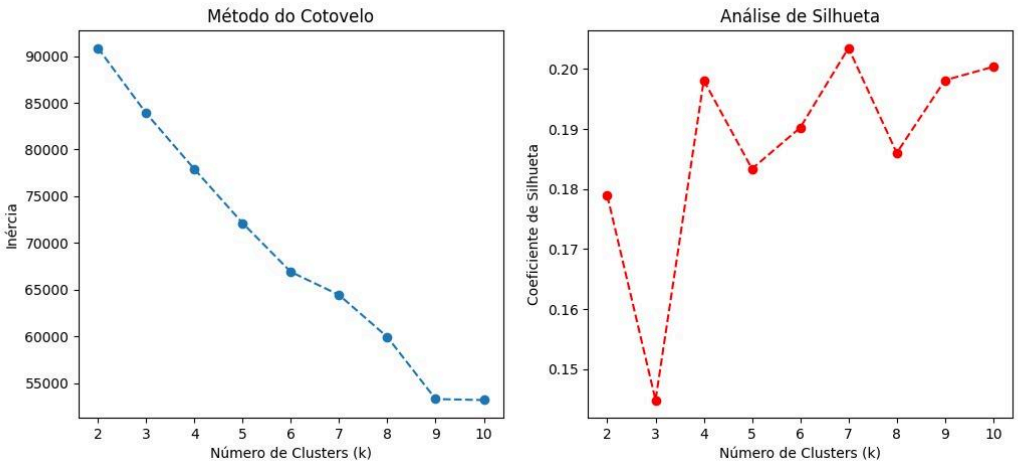
Criamos o gráfico de cotovelo para ver onde a inércia começa a diminuir menos rapidamente. Além disso, criamos um gráfico que mostra o coeficiente de silhueta, que quanto maior o valor, melhor o agrupamento.

Melhor número de clusters

```
best_k = K_range[np.argmax(silhouette_scores)]
print("Melhor número de clusters baseado na silhueta:", best_k)
```

Pega o valor de k que teve o maior coeficiente de silhueta (melhor qualidade de cluster). Após isso, o valor de k é exibido.

RESULTADOS



Método cotovelo

Ele mostra como a inércia, que é a soma das distâncias dos pontos ao centro do cluster, muda conforme aumentamos o número de clusters (k). Normalmente, procuramos o "cotovelo", onde a redução na inércia começa a desacelerar.

Coeficiente de silhueta

Mede a qualidade do agrupamento. valores mais altos indicam que o clusters mais bem definidos. O coeficiente de silhueta oscila bastante. O pico mais alto parece estar em **k = 7**, o que sugere que **7** clusters é uma boa escolha.

QUESTÃO 3

Escolha os dois atributos mais relevantes utilizando o método de Lasso e recalcule a quantidade de clusters com método do cotovelo e silhueta. Mudou a quantidade de cluster em relação à questão anterior? Se k diferentes para os dois métodos de determinação da quantidade de clusters, faça dois scatterplots e faça uma análise visual.

Importação das bibliotecas necessárias

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Lasso
from sklearn.metrics import silhouette_score
```

numpy: Usada para ordenação e seleção dos atributos e no cálculo de coeficiente.

pandas: Usada para manipulação de dados.

matpllotlib.pyplot: Usada para criar gráficos (método cotovelo e silhueta).

sklearn.cluster.KMeans: Para aplicar o algoritmo de clustering (agrupamento).

sklearn.preprocessing.StandardScaler: Usada para normalizar antes do agrupamento.

Sklearn.linear_model.Lasso: usada para identificar os atributos mais relevantes.

Sklearn.metrics.silhouette_score: Usada para determinar o melhor número de clusters.

Remoção da coluna alvo e tratamento de valores ausentes

```
X = df.drop(columns=["stroke"])
X = X.apply(lambda col: col.fillna(col.mean()) if col.dtype in ['int64', 'float64'] else col)
X = pd.get_dummies(X, drop_first=True)
```

Removemos a classe “stroke” pois os clusters são gerados sem considerar a classe alvo.

Para colunas numéricas, preenchemos os valores ausentes com a média .

Conversão de variáveis categóricas para numéricas .

Padronização dos dados

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Vai transformar os valores para uma distribuição normal com média 0 e desvio padrão 1. isso ajuda a melhorar o desempenho do algoritmo.

Seleção de atributos com o Lasso

```
lasso = Lasso(alpha=0.01)
lasso.fit(X_scaled, np.zeros(X_scaled.shape[0]))

coeficientes = np.abs(lasso.coef_)
atributos = np.array(X.columns)
indices_top2 = np.argsort(coeficientes)[-2:]
atributos_top2 = atributos[indices_top2]
```

A aplicação do Lasso vai selecionar os dois atributos mais importantes. **lasso.fit** faz com que o Lasso apenas selecione atributos e não treine um modelo preditivo.

Pegamos os valores absolutos dos coeficientes.
Pegamos os nomes das colunas.
Pegamos os dois maiores coeficientes.
Pegamos os nomes dos dois atributos escolhidos.

Cria um novo dataframe somente com dois atributos

```
X_selected = X[atributos_top2]
```

Aplicação do método cotovelo e silhueta

```
inertia = []
silhouette_scores = []
K_range = range(2, 11)

for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    clusters = kmeans.fit_predict(X_selected)
    inertia.append(kmeans.inertia_)

    if k > 1:
        silhouette_scores.append(silhouette_score(X_selected, clusters))
    else:
        silhouette_scores.append(0)
```

Armazenamos os valores da inércia.
Armazenamos os valores do coeficiente de silhueta.
Definimos o número de k(cluster) para testes.

O loop é usado e é iterado sobre cada valor de k (2 a 10).
Criamos um modelo Kmeans com k cluster.
n_cluster=k: define o número de clusters.
random_state=42: garante a reprodutibilidade dos resultados.
n_init=10: o algoritmo roda 10 vezes com diferentes inicializações.

O modelo é treinado e atribuímos cada ponto a um cluster.
Retornamos à soma das distâncias quadradas entre cada ponto e o centro do cluster. Valores menores indicam que os pontos estão mais próximos do seus centros.

if k > 1: Calcula o coeficiente de silhueta.

Identificação do melhor número de clusters

```
best_k_silhouette = K_range[np.argmax(silhouette_scores)]
print(f"Melhor número de clusters baseado na silhueta: {best_k_silhouette}")
```

O melhor k é aquele que maximiza o coeficiente de silhueta.

No nosso caso, o melhor k foi 3.

Plotagem dos gráficos

```
fig, axes = plt.subplots( nrows: 1,  ncols: 2, figsize=(12, 4))

axes[0].plot(K_range, inertia, 'o--b')
axes[0].set_title("Método do Cotovelo")
axes[0].set_xlabel("Número de Clusters (k)")
axes[0].set_ylabel("Inércia")

axes[1].plot(K_range, silhouette_scores, 'o--r')
axes[1].set_title("Análise de Silhueta")
axes[1].set_xlabel("Número de Clusters (k)")
axes[1].set_ylabel("Coeficiente de Silhueta")
```

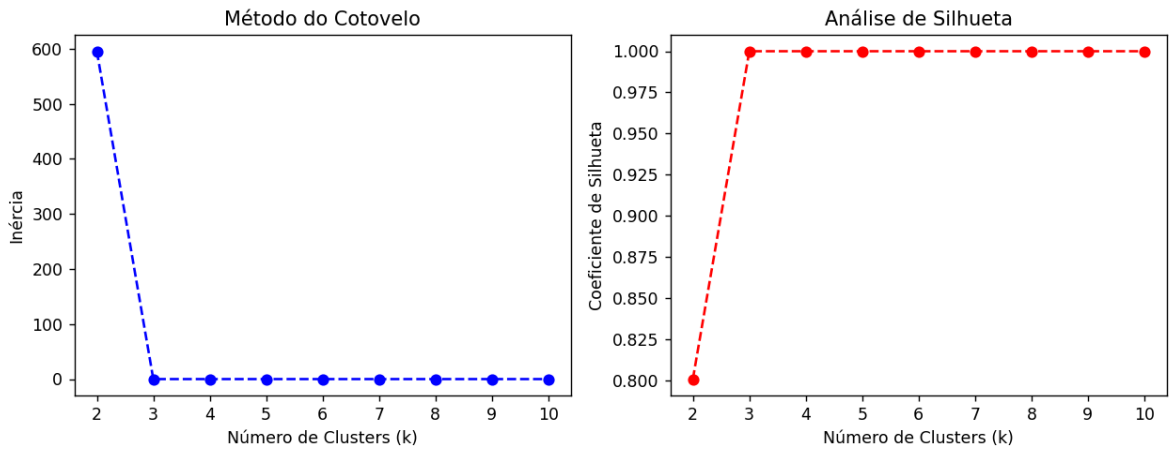
Criamos dois gráficos para visualizar os resultados.

Gráfico da Inércia (Método do Cotovelo) onde a curva "dobra" é o ponto ideal.
Gráfico da Silhueta mostra qual k é melhor.

RESULTADOS

```
Melhor número de clusters baseado na silhueta: 3
```

k = 3 apresentou o maior equilíbrio entre os dois critérios usados para determinar a quantidade ideal de clusters (cotovelo e silhueta)



As variáveis selecionadas foram (**'smoking_status_never smoked'** e **'smoking_status_smokes').** Portanto a segmentação dos clusters foram baseadas nessas informações.

Análise dos gráficos

Gráfico de cotovelo:
Para k=2 a inércia é muito alta.
Para k=3, há uma queda brusca na inércia.
Para k>3, a redução da inércia **fica quase constante**, indicando que adicionar mais clusters não melhora significativamente a compactação dos grupos.

Gráfico de silhueta:
Para k=2, a silhueta ainda está baixa.
Para k=3, a silhueta aumenta significativamente, indicando que os clusters estão bem definidos.
Para k>3, os valores se estabilizam e não há um aumento significativo.

Conclusão: Ambos os métodos indicam que o valor ideal é **3**.

Questão 4

Faça um crosstab para que possamos identificar como ficou a distribuição de cluster de acordo com classes presentes na coluna alvo do seu dataset. Utilize o K means e utilize o k obtido pelo indice de silhueta.

Importando as bibliotecas necessárias

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
```

pandas: Manipula dados tabulares, como CSVs, permitindo filtragem, análise e visualização de tabelas.

`numpy`: Fornece operações matemáticas eficientes, como arrays e cálculos numéricos avançados.

`matplotlib.pyplot`: Cria gráficos e visualizações, permitindo personalizar e exibir dados visualmente.

`seaborn`: Facilita a criação de gráficos estatísticos mais bonitos e informativos com menos código.

`sklearn.cluster.KMeans`: Algoritmo de agrupamento que separa dados em k grupos com base em similaridades.

`sklearn.preprocessing.StandardScaler`: Padroniza os dados para ter média 0 e desvio padrão 1, melhorando a precisão do modelo.

`sklearn.metrics.silhouette_score`: Mede a qualidade dos agrupamentos feitos pelo K-Means, avaliando a separação dos clusters.

Selecionando os atributos importantes e preparando os dados

```
atributos_relevantes = ["age", "hypertension", "heart_disease", "avg_glucose_level", "bmi"]
```

São escolhidas as colunas que serão usadas para formar os grupos:

age (idade)

hypertension (hipertensão)

heart_disease (doença cardíaca)

avg_glucose_level (nível médio de glicose no sangue)

bmi (Índice de Massa Corporal)

```
X = df[atributos_relevantes]
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Selecionamos os dados das colunas escolhidas e após isso padronizamos os valores para que todas as variáveis tenham a mesma escala (média 0 e desvio padrão 1), o que melhora a precisão do algoritmo.

Aplicando o K-Means

```
k = 3
kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
df["Cluster"] = kmeans.fit_predict(X_scaled)
```

Definimos que serão criados 3 grupos (k=3), com isso, executamos o K-Means, que agrupa os pacientes com base nas semelhanças entre as variáveis escolhidas.

Depois, adiciona uma nova coluna "Cluster" no DataFrame, indicando a qual grupo cada paciente pertence.

Analisando a relação entre os grupos e AVC

```
crosstab_result = pd.crosstab(df["Cluster"], df["stroke"])
print("\nTabela de Contingência (Crosstab):")
print(crosstab_result)
```

Criamos uma tabela cruzada que mostra quantos pacientes em cada grupo tiveram ou não AVC (stroke). Isso ajuda a entender se algum grupo tem maior risco de AVC.

Visualização dos Clusters

```
plt.figure(figsize=(10, 6))
sns.scatterplot(x=df["age"], y=df["avg_glucose_level"], hue=df["Cluster"], palette="viridis", s=100, alpha=0.7)
```

Criamos um gráfico de dispersão, onde:

Eixo X: idade dos pacientes (age).

Eixo Y: nível médio de glicose no sangue (avg_glucose_level).

Os pontos são coloridos de acordo com o cluster.

```
centroids = kmeans.cluster_centers_
for i, centroid in enumerate(centroids):
    plt.scatter(centroid[0], centroid[3], marker='X', s=200, c='red', label=f'Centroid {i+1}' if i == 0 else "")
```

Adicionamos os centróides que são os pontos centrais de cada grupo.

Como existem várias variáveis, ele escolhe a média dos valores para posicionar os centróides.

```
plt.title( label: 'Distribuição dos Pacientes com Base nos Clusters (Age vs Avg Glucose Level) com Centroids', fontsize=16)
plt.xlabel( xlabel: 'Age', fontsize=12)
plt.ylabel( ylabel: 'Avg Glucose Level', fontsize=12)
plt.legend(title='Cluster', loc='upper right')
plt.show()
```

Adiciona título, rótulos e legenda ao gráfico.

Exibe o gráfico final.

Resultados Obtidos

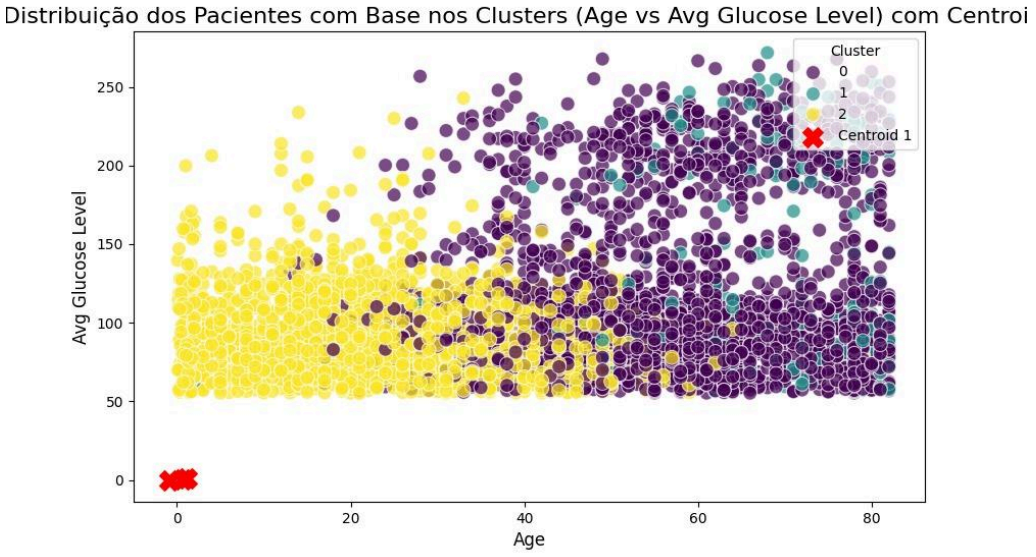
```
Tabela de Contingência (Crosstab):
stroke      0    1
Cluster
0        2344  161
1         203   40
2        2153    8
```

Essa tabela mostra quantos pacientes pertencem a cada cluster e quantos deles tiveram ou não AVC (stroke).

Cluster	Não tiveram AVC (0)	Tiveram AVC (1)
0	2344	161
1	203	40
2	2153	8

Isso demonstra que, o Cluster 1 pode estar mais associado ao AVC, já que a proporção de pacientes com AVC é maior (40 em 243 pacientes ≈ 16,5%). E o Cluster 2 tem muito menos casos de AVC em comparação aos outros grupos, indicando um perfil de pacientes com menor risco.

Resultado do Gráfico



Esse gráfico mostra os pacientes distribuídos por idade e nível médio de glicose no sangue, agrupados pelos clusters gerados pelo K-Means.

Assim os pacientes estão separados em 3 grupos (clusters):

Cluster 0 (roxo)

Cluster 1 (azul esverdeado)

Cluster 2 (amarelo)

Os pontos vermelhos representam o centro (centroid) de um cluster:

Distribuição dos grupos:

O **Cluster 2 (amarelo)** se concentra mais em pacientes mais jovens e com níveis de glicose mais baixos.

O **Cluster 0 (roxo)** parece englobar uma grande variedade de idades e níveis de glicose.

O **Cluster 1 (azul esverdeado)** aparece em menor quantidade, o que condiz com a tabela de contingência, onde ele tem menos pacientes.

4. Conclusões

Nesta análise, realizamos um estudo de clusterização utilizando o algoritmo K-Means para segmentar os dados do dataset de saúde, tendo como referência a coluna alvo 'stroke'. O dataset usado diz respeito à previsão de Acidente Vascular Cerebral (AVC). Realizamos também a determinação do melhor número de clusters através dos algoritmos. A clusterização permitiu identificar padrões dentro do conjunto de dados, mostrando que os pacientes podem ser agrupados de forma coerente de acordo com suas características. Além disso, realizamos uma análise das questões e dos resultados através de gráficos.

5. Próximos passos

Testar outros algoritmos de clusterização

Utilização dos clusters para ajudar na predição ajudando a aprimorar a previão de AVC.